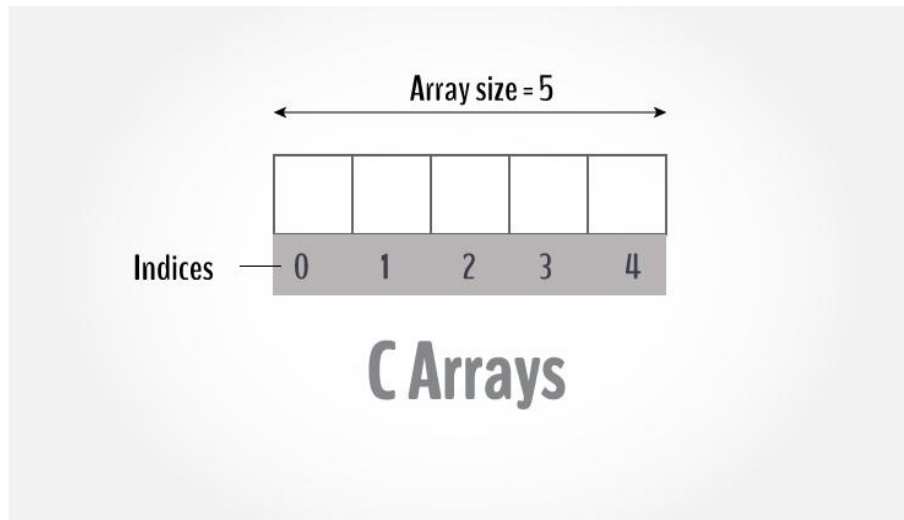


➤ What is an Array in C :-

1. Array is the collection of same type of Data Items. It is Homogeneous type of data.
2. Array is Always start from 0 index and end will size -1.
3. It's a Fixed sized, We cannot Increase and Decrease at the runtime.
4. Index through we can Access that data easily, without index we cannot access that data.



• Declaration of Array :-

```
data_type array_name[array_size];
```

Example:-

```
int a[5];
```

• Advantages of Array :-

1. Easy of Searching.
2. Easy of Sorting.
3. Random Access.
4. Code Optimization.

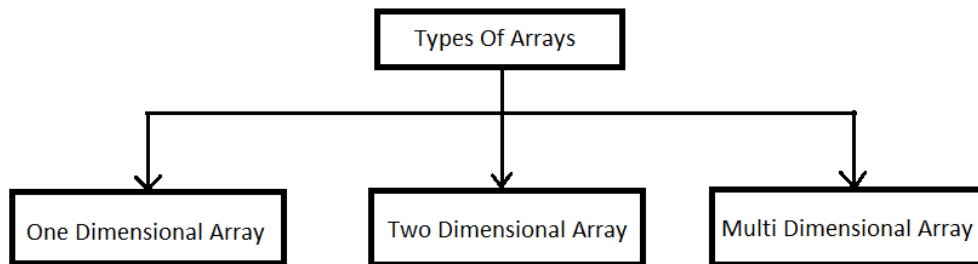
• Disadvantages of Array:-

1. We Cannot Store Different type of Data.
2. Fixed Size.
3. Insertion and Deletion array is Worst Case.
4. Waste of Momery.

➤ Types of Array :-

They are primarily categorized based on their dimensions and how their memory is handled.

There are three types of array.



1) One Dimensional Array (1D) :-

1. A 1D array is a linear collection of elements stored in contiguous memory locations.
2. Each element is accessed using a single index, starting from 0.
3. All elements in the array must be of the same data type (e.g., int, float).
4. It allows efficient storage and access to a fixed set of related data, like marks or temperatures.

Syntax :-

```
datatype array_name[size];
```

Example :- `int numbers[5];` declares an array named numbers that can hold 5 integer values.

```
int num[3];  
num[0]=10;//initialization of array  
num[1]=20;  
num[2]=30;  
printf("%d",num[0]);
```

```
printf("%d",num[1]);
```

```
printf("%d",num[2]);
```

OR

```
int i;
```

```
for(i=0;i<3;i++)
```

```
{
```

```
printf("%d ",num[i]);
```

```
}
```

2) Two Dimensional Array (2D) :-

1. A 2D array is an array of arrays, often used to represent matrices or tables.
2. Elements are accessed using two indices: one for row and one for column.
3. Memory is allocated in row-major order, meaning rows are stored one after another in memory.
4. All elements must be of the same data type, and the dimensions must be fixed at compile-time.

Syntax :-

```
datatype array_name[rows][columns];
```

Example :- `int matrix[3][4];` declares a 2D array with 3 rows and 4 columns.

```
int num[2][2]={{1,2},{3,4}};
```

```
int i,j;
```

```
for(i=0;i<2;i++)
```

```
{
```

```
for(j=0;j<2;j++)
```

```
{
```

```
Print("%d",num[i][j]);
```

```
}
```

```
}
```

3) Three Dimensional Array (3D) :-

1. A 3D array is an array of 2D arrays, useful for representing data in a cube or layered format (like 3D grids).
2. Elements are accessed using three indices: [depth][row][column].
3. It stores data in contiguous memory locations, managed in row-major order.
4. Commonly used in applications like image processing, simulations, and 3D modeling.

Syntax :-

```
datatype array_name[x][y][z];
```

Example :- `int cube[3][3][3];` declares a 3D array.

```
int num[2][2][2];
```

```
int i,j,k;
```

```
for(i=0;i<2;i++)
```

```
{
```

```
for(j=0;j<2;j++)
```

```
{
```

```
for(k=0;k<2;k++)
```

```
{
```

```
Printf("%d", num[i][j][k]);
```

```
}
```

```
}
```

```
}
```